National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Faculty of Computing | BSCS-2023-CD

CS-346: Compiler Construction

Lab 11: Code Optimization

Loop & IR Optimization Techniques

Date: 23rd April 2026
Lab Instructor: Ms. Urooj Akmal
Instructor: Miss Yusra
Name (Qalam ID): Abrar Butt (460297)
GitHub: github.com/Abrarbut/compiler-construction

Contents

Introduction **2**

1 Modularize the Compiler **3**
1.1 Compiler Architecture . . . 3
1.2 Key Design Decisions . . . 3
1.3 Token Types Recognised by the Lexer . . . 3

2 IR Generation (Three Address Code) **4**
2.1 TAC Format . . . 4
2.2 Sample Input Program . . . 4
2.3 Generated Unoptimized IR . . . 4
2.4 IR Generation Algorithm . . . 4

3 Code Optimization Passes **6**
3.1 Constant Folding . . . 6
3.2 Constant Propagation . . . 6
3.3 Algebraic Simplification . . . 7
3.4 Common Subexpression Elimination (CSE) . . . 7
3.5 Optimized IR After All Four Passes . . . 8

4 Dead and Unreachable Code Elimination **9**
4.1 Types of Eliminated Code . . . 9
4.2 Implementation . . . 9
4.3 Output . . . 10

5 Loop Optimizations **11**
5.1 Sample Loop Program . . . 11
5.2 Unoptimized Loop IR . . . 11
5.3 Loop Invariant Code Motion (LICM) . . . 11
5.4 Loop Fusion . . . 12
5.5 Strength Reduction . . . 12
5.6 Optimized Loop IR After All Three Passes . . . 13

6 Execution Time Comparison **14**
6.1 Benchmark Design . . . 14
6.2 Results . . . 14
6.3 Analysis . . . 14

Full Source Code **16**

Program Output **27**

Summary and Conclusions **30**

Introduction

Code Optimization is a program transformation technique that improves program performance by reducing resource usage (CPU, memory) and increasing execution speed, while preserving correctness.

Three Rules of an Optimizing Compiler

1. The optimized code **must not change** the program's meaning.
2. It should **improve execution speed** and/or reduce resource usage.
3. The optimization process itself must be **efficient** and not delay compilation significantly.

Loops consume up to 90% of execution time in real programs, making loop optimization the highest-leverage target. Since many optimization problems are NP-complete, practical compilers use *heuristic passes*.

Objectives:

1. Implement and analyze **code optimization** techniques.
2. Implement and evaluate **loop optimization** techniques.
3. Compare performance **before and after** optimization.
4. Understand the compiler's role in improving efficiency.

Code Optimizations Implemented

- Constant Folding
- Constant Propagation
- Algebraic Simplification
- Common Subexpression Elimination
- Unreachable Code Elimination
- Dead Variable Elimination

Loop Optimizations Implemented

- Loop Invariant Code Motion (LICM)
- Loop Fusion
- Strength Reduction

Task 1: Modularize the Compiler

Refactor the compiler into clear, modular stages ready for optimization. Ensure clean code structure, proper variable handling, and removal of redundant logic.

1.1 Compiler Architecture

The compiler is divided into five independent classes, each owning one pipeline stage:

Stage	Class	Responsibility
Lexical Analysis	`Lexer`	Source text → token stream
Parsing + IR Gen	`IRGenerator`	Tokens → TAC instructions
Code Optimization	`Optimizer`	CF, CP, AS, CSE passes
Dead Code Elim.	`DeadCodeEliminator`	Remove dead/unreachable code
Loop Optimization	`LoopOptimizer`	LICM, Fusion, Strength Reduction

1.2 Key Design Decisions

- Every optimization pass is a **pure function**: `vector<TACInstruction>` in, modified vector out – trivially chainable.
- `TACInstruction` uses boolean flags (`isReturn`, `isLoopHead`, `isLoopEnd`, ...) so no string parsing is needed during optimization passes.
- The `raw` field caches the human-readable instruction string, rebuilt only when the instruction changes.

1.3 Token Types Recognised by the Lexer

Type	**Examples**
`KEYWORD`	`int`, `float`, `return`, `for`, `while`
`IDENT`	`x`, `sum`, `total`
`NUMBER`	`0`, `42`, `100`
`OP`	`+ - * / %`
`ASSIGN`	`=`
`RELOP`	`< > <= >= == !=`
`SEMI`	`;`
`PAREN`	`(  )`
`BRACE`	`{  }`

Task 2: IR Generation (Three Address Code)

Extend the compiler to generate **Three Address Code (TAC)** and print the IR *before* any optimization.

2.1 TAC Format

Each TAC instruction has at most two operands and one result:

$$\texttt{result} = \texttt{arg}_1 \; op \; \texttt{arg}_2$$

Temporaries `t1`, `t2`, ... hold intermediate values.

2.2 Sample Input Program

```
int compute(int a) {
    int x = 3 + 4;
    int y = x * 1;
    int z = y + 0;
    int w = z * 2;
    int t = z * 2;    // same as w -- CSE candidate
    return t;
    int dead = 99;    // unreachable code -- Task 4 target
}
```

Listing 1: Input source – Tasks 2 to 4

2.3 Generated Unoptimized IR

```
t1 = 3 + 4
x  = t1
t2 = x * 1
y  = t2
t3 = y + 0
z  = t3
t4 = z * 2
w  = t4
t5 = z * 2      ; duplicate -- CSE candidate
t  = t5
return t
dead = 99       ; unreachable -- eliminated in Task 4
```

2.4 IR Generation Algorithm

`IRGenerator::generateIR()` tokenizes the source then uses a **recursive descent parser**:

$$\texttt{parseExpr} \rightarrow \texttt{parseTerm} \rightarrow \texttt{parseFactor}$$

Each binary operation allocates a fresh temporary via `newTemp()` and calls `emit(result, op, arg1, arg2)`. `for`/`while` heads emit `LOOP_BEGIN` / `LOOP_END` markers consumed by the loop optimizer.

Task 3: Code Optimization Passes

Implement at least 3–5 optimization techniques on the IR. Show IR before and after each transformation with explanatory comments.

Four passes run in sequence:

$$\text{Constant Folding} \rightarrow \text{Constant Propagation} \rightarrow \text{Algebraic Simplification} \rightarrow \text{CSE}$$

3.1 Constant Folding

Idea: If both operands are numeric literals, evaluate at compile time.

$$\texttt{t1 = 3 + 4} \implies \texttt{t1 = 7}$$

```
// [CF] If both operands are literal numbers, compute at compile time
if (!ins.op.empty() && isNumber(ins.arg1) && isNumber(ins.arg2)) {
    long long a = stoll(ins.arg1), b = stoll(ins.arg2), res;
    if      (ins.op == "+") res = a + b;
    else if (ins.op == "-") res = a - b;
    else if (ins.op == "*") res = a * b;
    else if (ins.op == "/" && b != 0) res = a / b;
    ins.arg1 = to_string(res);
    ins.op = ""; ins.arg2 = "";
    ins.raw = ins.result + " = " + ins.arg1;
}
```

Listing 2: Constant Folding

3.2 Constant Propagation

Idea: Track variables that hold known constants; substitute them wherever they appear as operands.

$$\texttt{x = 7;}\quad \texttt{t2 = x * 1} \implies \texttt{t2 = 7 * 1}$$

```
// [CP] Substitute known constant values for variable uses
map<string, string> constMap;
for (auto& ins : ir) {
    if (constMap.count(ins.arg1)) ins.arg1 = constMap[ins.arg1];
    if (constMap.count(ins.arg2)) ins.arg2 = constMap[ins.arg2];
    if (ins.op.empty() && isNumber(ins.arg1))
        constMap[ins.result] = ins.arg1;   // record: result is a constant
    else
        constMap.erase(ins.result);        // invalidate: no longer constant
}
```

Listing 3: Constant Propagation

3.3 Algebraic Simplification

Idea: Apply algebraic identities to remove redundant operations.

Pattern	Simplifies to	Rule
$x \times 1$	x	Multiplicative identity
$1 \times x$	x	Commutativity
$x \times 0$	0	Zero product
$x + 0$	x	Additive identity
$x - 0$	x	Subtraction identity
$x \div 1$	x	Division identity

```
// [AS] Replace identity / zero-product patterns
if (ins.op == "*") {
    if (ins.arg2=="1")            { newArg=ins.arg1; simplified=true; }
    if (ins.arg1=="1")            { newArg=ins.arg2; simplified=true; }
    if (ins.arg2=="0"||ins.arg1=="0") { newArg="0"; simplified=true; }
}
if (ins.op=="+" && ins.arg2=="0") { newArg=ins.arg1; simplified=true; }
if (ins.op=="-" && ins.arg2=="0") { newArg=ins.arg1; simplified=true; }
if (ins.op=="/" && ins.arg2=="1") { newArg=ins.arg1; simplified=true; }
if (simplified) { ins.arg1=newArg; ins.op=""; ins.arg2=""; }
```

Listing 4: Algebraic Simplification

3.4 Common Subexpression Elimination (CSE)

Idea: If `arg1 op arg2` was already computed into a temporary, reuse that temporary.

$$\texttt{t4 = z * 2;} \quad \texttt{t5 = z * 2} \implies \texttt{t4 = z * 2;} \quad \texttt{t5 = t4}$$

```
// [CSE] Reuse result of previously computed identical expression
map<string, string> exprMap; // key = "arg1 op arg2"
for (auto& ins : ir) {
    if (ins.op.empty()) continue;
    string key = ins.arg1 + " " + ins.op + " " + ins.arg2;
    if (exprMap.count(key)) {
        ins.arg1 = exprMap[key]; ins.op = ""; ins.arg2 = "";
        ins.raw  = ins.result + " = " + ins.arg1;
    } else {
        exprMap[key] = ins.result;
    }
}
```

Listing 5: CSE

3.5 Optimized IR After All Four Passes

```
t1 = 7            ; [CF]  3+4 folded to 7 at compile time
x  = 7            ; [CP]  x receives folded constant 7
t2 = 7            ; [CP+CF] x propagated as 7, then 7*1 folded
y  = t2
t3 = y            ; [AS]  y+0 simplified to y
z  = t3
t4 = z * 2        ; kept  (z is not a compile-time constant)
w  = t4
t5 = t4           ; [CSE] z*2 already computed as t4 -- reused
t  = t5
return t
dead = 99         ; removed in Task 4
```

Task 4: Dead and Unreachable Code Elimination

Enhance the compiler with basic control-flow analysis to identify and remove unreachable statements and dead variable assignments.

4.1 Types of Eliminated Code

Unreachable Code: Any statement following a `return` (before the next label) can never execute. The `eliminate()` pass sets a flag on seeing `isReturn = true` and drops all subsequent instructions.

Dead Variable: A variable assigned a value that is *never read*. `deadVarElimination()` counts each variable's uses; if a temporary has zero uses, its defining instruction is removed.

4.2 Implementation

```
// [Unreachable] Drop all instructions after a return
bool unreachable = false;
for (auto& ins : ir) {
    if (ins.isLabel) { unreachable = false; result.push_back(ins); continue; }
    if (unreachable) { /* dropped */ continue; }
    result.push_back(ins);
    if (ins.isReturn) unreachable = true;
}
```

Listing 6: Unreachable code elimination

```
// [DeadVar] Count all variable uses; drop defs whose result is never read
map<string, int> useCount;
for (auto& ins : ir) {
    if (!ins.arg1.empty() && !isNumber(ins.arg1)) useCount[ins.arg1]++;
    if (!ins.arg2.empty() && !isNumber(ins.arg2)) useCount[ins.arg2]++;
    if (ins.isReturn) useCount[ins.arg1]++;
}
for (auto& ins : ir) {
    if (ins.result[0]=='t' && useCount[ins.result]==0) continue; // dropped
    result.push_back(ins);
}
```

Listing 7: Dead variable elimination

4.3 Output

```
[Dead Code Eliminated]: dead = 99     <- after return:
   unreachable
[Dead Var Eliminated]:  t1 = 7        <- t1 never read
```

```
x  = 7
t2 = 7
y  = t2
t3 = y
z  = t3
t4 = z * 2
w  = t4
t5 = t4
t  = t5
return t
```

Task 5: Loop Optimizations

Extend the compiler to detect and optimize loops. Implement Loop Invariant Code Motion, Loop Fusion, and Strength Reduction.

5.1 Sample Loop Program

```
int loopFunc() {
    int base = 10 * 2;              // constant folding candidate
    for (i = 0; i < 100; i++) {
        sum    = sum + i;           // depends on i -- stays in loop
        stride = base * 3;         // LOOP INVARIANT: base never
   changes!
    }
    for (j = 0; j < 100; j++) {    // FUSION candidate: same trip count
        total = total + j;
    }
    return sum;
}
```

Listing 8: Loop source – Task 5

5.2 Unoptimized Loop IR

```
t1          = 10 * 2
base        = t1
LOOP_BEGIN  LOOP_1
  i         = 0
  t2        = sum + i
  sum       = t2
  t3        = base * 3    ; invariant -- base unchanged
  stride    = t3
LOOP_END    LOOP_1
LOOP_BEGIN  LOOP_1         ; 2nd loop -- fusion candidate
  j         = 0
  t4        = total + j
  total     = t4
LOOP_END    LOOP_1
return sum
```

5.3 Loop Invariant Code Motion (LICM)

Idea: Move instructions whose operands are never modified inside the loop to just *before* the loop, eliminating redundant recomputation on every iteration.

Validity Conditions:

1. Neither `arg1` nor `arg2` is assigned inside the loop.
2. The instruction has no side effects.

In our example `t3 = base * 3` qualifies because `base` is never written inside `LOOP_1`.

```
// [LICM] Collect all variables modified in loop body
set<string> modified;
for (auto& ins : loopBody) modified.insert(ins.result);

// Hoist instructions whose operands are not in modified set
for (auto& ins : loopBody) {
    bool invariant = !ins.op.empty()
                    && !modified.count(ins.arg1)
                    && (ins.arg2.empty() || !modified.count(ins.arg2));
    if (invariant) hoisted.push_back(ins);
    else            remaining.push_back(ins);
}
// Output: [hoisted code] LOOP_BEGIN [remaining body] LOOP_END
```

Listing 9: LICM hoisting

5.4 Loop Fusion

Idea: Merge two consecutive loops with the same trip count into one loop, cutting loop-control overhead in half and improving cache locality.

Validity Conditions:

1. Both loops iterate over the **same range** (same bounds).
2. No data dependence from loop 2 reads a value written by loop 1 in an order-sensitive way.
3. No intervening code with side effects between the two loops.

```
// [Fusion] Detect two back-to-back LOOP_BEGIN..LOOP_END regions
if (end1 + 1 < ir.size() && ir[end1 + 1].isLoopHead) {
    // Same trip count -> valid to fuse
    // Emit: one LOOP_BEGIN, body1 + body2, one LOOP_END
    result.push_back(fusedHead);
    for (auto& b : body1) result.push_back(b);
    for (auto& b : body2) result.push_back(b);
    result.push_back(fusedTail);
}
```

Listing 10: Loop Fusion

5.5 Strength Reduction

Idea: Replace expensive multiplications with cheaper additions. For an induction variable i multiplied by constant c, maintain an accumulator incremented by c each iteration.

$$\underbrace{i \times c}_{\text{multiply each iter}} \implies \underbrace{\texttt{acc} \mathrel{+}= c;\ \ t = \texttt{acc}}_{\text{add each iter}}$$

Applied to `10 * 2`:

$$\texttt{t1 = 10 * 2} \implies \texttt{t1 = 10 + 10}$$

```
// [SR] x * 2  -->  x + x  (one add beats one multiply every time)
if (ins.op == "*" && isNumber(ins.arg2) && stoll(ins.arg2) == 2) {
    ins.op   = "+";
    ins.arg2 = ins.arg1;
    ins.raw  = ins.result + " = " + ins.arg1 + " + " + ins.arg2;
}
```

Listing 11: Strength Reduction: mul-by-2 to add

5.6 Optimized Loop IR After All Three Passes

```
t1              = 10 + 10        ; [SR]   10*2 -> 10+10
base            = t1
t3              = base * 3       ; [LICM] hoisted out of loop
LOOP_BEGIN      LOOP_1_FUSED     ; [Fusion] single merged loop
  i             = 0
  t2            = sum + i
  sum           = t2
  stride        = t3             ; uses hoisted invariant
  j             = 0
  t4            = total + j
  total         = t4
LOOP_END        LOOP_1_FUSED
return sum
```

Task 6: Execution Time Comparison

Run both optimized and unoptimized versions multiple times and display execution times before and after optimization.

6.1 Benchmark Design

Both versions compute $\sum_{i=0}^{N-1} 2i$ for $N = 50{,}000{,}000$, averaged over 5 runs using `std::chrono::high_resolu`

- **Unoptimized:** multiplies `i * factor` every iteration (factor re-loaded from stack each time).
- **Optimized (Strength Reduced):** accumulator `acc` incremented by 2 each step – zero multiplications.

```
// UNOPTIMIZED -- one multiply per iteration
volatile long long sum = 0;
for (int i = 0; i < N; ++i) {
    int factor = 2;          // re-evaluated every iteration
    sum += i * factor;
}

// OPTIMIZED -- strength reduction: add only
volatile long long sum = 0;
long long acc = 0;
for (int i = 0; i < N; ++i) {
    sum += acc;              // addition only -- no multiply
    acc += 2;
}
```

Listing 12: Benchmark kernel

6.2 Results

```
Unoptimized avg time :  128.701 ms
Optimized   avg time :   38.991 ms
Speedup              :    3.287x
(Both produce sum = 2499999950000000)
```

6.3 Analysis

Version	Avg Time (ms)	Op per Iteration	Speedup
Unoptimized	128.7	MUL + LOAD	1.00×
Optimized	39.0	ADD	≈ 3.3×

The $\approx 3.3\times$ speedup comes from:

1. Replacing a 3–5 cycle integer multiply with a 1-cycle integer add.
2. Eliminating the redundant stack-load of `factor` each iteration.
3. Shorter critical-path depth enables the CPU pipeline to issue more iterations per clock.

Full Source Code

Complete single-file implementation. Also available on GitHub:

https://github.com/Abrarbut/compiler-construction.git

```


#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <sstream>
#include <chrono>
#include <algorithm>
#include <iomanip>

// ================================================================
// DATA STRUCTURES
// ================================================================

struct Token {
    std::string type;
    std::string value;
};

struct TACInstruction {
    std::string result, op, arg1, arg2;
    bool isLabel    = false;
    bool isJump     = false;
    bool isReturn   = false;
    bool isLoopHead = false;
    bool isLoopEnd  = false;
    std::string raw;
};

// ================================================================
// TASK 1: LEXER
// ================================================================

class Lexer {
public:
    std::vector<Token> tokenize(const std::string& src) {
        std::vector<Token> tokens;
        size_t i = 0;
        while (i < src.size()) {
            if (std::isspace(src[i])) { ++i; continue; }
            // Single-line comment
            if (i+1 < src.size() && src[i]=='/' && src[i+1]=='/') {
                while (i < src.size() && src[i] != '\n') ++i;
                continue;
            }
            // Number
            if (std::isdigit(src[i])) {
                std::string num;
                while (i < src.size() && std::isdigit(src[i])) num +=
    src[i++];
```

```
                tokens.push_back({"NUMBER", num}); continue;
            }
            // Identifier / keyword
            if (std::isalpha(src[i]) || src[i] == '_') {
                std::string word;
                while (i<src.size() && (std::isalnum(src[i])||src[i]==
    '_'))
                    word += src[i++];
                static const std::set<std::string> kw =
                    {"int","float","return","if","else","for","while",
    "void"};
                tokens.push_back({kw.count(word)?"KEYWORD":"IDENT",
    word});
                continue;
            }
            // Two-char operators
            if (i+1 < src.size()) {
                std::string two = {src[i], src[i+1]};
                if (two=="==" || two=="!=" || two=="<=" || two==">=")
    {
                    tokens.push_back({"RELOP", two}); i += 2; continue
    ;
                }
            }
            // Single-char
            std::string ch(1, src[i]);
            if (ch=="+"||ch=="-"||ch=="*"||ch=="/"||ch=="%")
                tokens.push_back({"OP", ch});
            else if (ch=="=")  tokens.push_back({"ASSIGN", ch});
            else if (ch==";")  tokens.push_back({"SEMI",   ch});
            else if (ch=="("||ch==")") tokens.push_back({"PAREN", ch})
    ;
            else if (ch=="{"||ch=="}") tokens.push_back({"BRACE", ch})
    ;
            else if (ch=="<"||ch==">") tokens.push_back({"RELOP", ch})
    ;
            else if (ch==",")  tokens.push_back({"COMMA",  ch});
            else               tokens.push_back({"UNKNOWN",ch});
            ++i;
        }
        return tokens;
    }

    void printTokens(const std::vector<Token>& toks) {
        std::cout << "\n=== LEXER OUTPUT ===\n";
        for (auto& t : toks)
            std::cout << "  [" << std::setw(8) << t.type << "] "
                      << t.value << "\n";
    }
};

// ==============================================================
// TASK 1 & 2: PARSER + IR GENERATOR
// ==============================================================

class IRGenerator {
    int tempCount = 0;
    std::string newTemp() { return "t"+std::to_string(++tempCount); }
```

```
public:
    std::vector<TACInstruction> instructions;

    void emit(const std::string& res, const std::string& op,
              const std::string& a1, const std::string& a2 = "") {
        TACInstruction i;
        i.result=res; i.op=op; i.arg1=a1; i.arg2=a2;
        i.raw = res+" = "+a1;
        if (!op.empty()) i.raw += " "+op+" "+a2;
        instructions.push_back(i);
    }
    void emitReturn(const std::string& val) {
        TACInstruction i; i.isReturn=true; i.arg1=val;
        i.raw = "return "+val; instructions.push_back(i);
    }
    void emitLoopHead(const std::string& lbl) {
        TACInstruction i; i.isLoopHead=true; i.result=lbl;
        i.raw = "LOOP_BEGIN "+lbl; instructions.push_back(i);
    }
    void emitLoopEnd(const std::string& lbl) {
        TACInstruction i; i.isLoopEnd=true; i.result=lbl;
        i.raw = "LOOP_END "+lbl; instructions.push_back(i);
    }

    std::string parseFactor(const std::vector<Token>& t, size_t& p) {
        if (p<t.size() && t[p].type=="PAREN" && t[p].value=="(") {
            ++p;
            std::string r = parseExpr(t, p);
            if (p<t.size() && t[p].value==")") ++p;
            return r;
        }
        if (p<t.size() && (t[p].type=="NUMBER"||t[p].type=="IDENT"))
            return t[p++].value;
        return "0";
    }
    std::string parseTerm(const std::vector<Token>& t, size_t& p) {
        std::string L = parseFactor(t, p);
        while (p<t.size() && t[p].type=="OP" &&
               (t[p].value=="*"||t[p].value=="/")) {
            std::string op = t[p++].value;
            std::string R  = parseFactor(t, p);
            std::string tmp = newTemp();
            emit(tmp, op, L, R); L = tmp;
        }
        return L;
    }
    std::string parseExpr(const std::vector<Token>& t, size_t& p) {
        std::string L = parseTerm(t, p);
        while (p<t.size() && t[p].type=="OP" &&
               (t[p].value=="+"||t[p].value=="-")) {
            std::string op = t[p++].value;
            std::string R  = parseTerm(t, p);
            std::string tmp = newTemp();
            emit(tmp, op, L, R); L = tmp;
        }
        return L;
    }

```

```
    void generateIR(const std::string& source) {
        Lexer lx; auto tokens = lx.tokenize(source);
        size_t pos = 0; int loopCount = 0;
        while (pos < tokens.size()) {
            // Type keyword
            if (tokens[pos].type=="KEYWORD" &&
                (tokens[pos].value=="int"||tokens[pos].value=="float" ||
                 tokens[pos].value=="void")) {
                ++pos;
                if (pos+1<tokens.size() && tokens[pos].type=="IDENT" &&
                    tokens[pos+1].value=="(") {
                    ++pos;
                    while (pos<tokens.size() && tokens[pos].value!=")") ++pos;
                    if (pos<tokens.size()) ++pos;
                }
                continue;
            }
            // return
            if (tokens[pos].type=="KEYWORD" && tokens[pos].value=="return") {
                ++pos;
                if (pos<tokens.size() && tokens[pos].type!="SEMI")
                    emitReturn(parseExpr(tokens, pos));
                else emitReturn("");
                if (pos<tokens.size() && tokens[pos].type=="SEMI") ++pos;
                continue;
            }
            // for loop
            if (tokens[pos].type=="KEYWORD" && tokens[pos].value=="for") {
                ++pos;
                if (pos<tokens.size() && tokens[pos].value=="(") ++pos;
                std::string lbl = "LOOP_"+std::to_string(++loopCount);
                emitLoopHead(lbl);
                if (pos<tokens.size() && tokens[pos].type=="IDENT") {
                    std::string var = tokens[pos++].value;
                    if (pos<tokens.size() && tokens[pos].type=="ASSIGN") ++pos;
                    std::string val = parseExpr(tokens, pos);
                    emit(var, "", val);
                    if (pos<tokens.size() && tokens[pos].type=="SEMI") ++pos;
                }
                while (pos<tokens.size() && tokens[pos].type!="SEMI") ++pos;
                if (pos<tokens.size()) ++pos;
                while (pos<tokens.size() && tokens[pos].value!=")") ++pos;
                if (pos<tokens.size()) ++pos;
                if (pos<tokens.size() && tokens[pos].value=="{") ++pos;
                continue;
            }
```

```
            // closing brace
            if (tokens[pos].type=="BRACE" && tokens[pos].value=="}") {
                if (loopCount > 0)
                    emitLoopEnd("LOOP_"+std::to_string(loopCount--));
                ++pos; continue;
            }
            // assignment
            if (tokens[pos].type=="IDENT") {
                std::string var = tokens[pos++].value;
                if (pos<tokens.size() && tokens[pos].type=="ASSIGN") {
                    ++pos;
                    emit(var, "", parseExpr(tokens, pos));
                }
                if (pos<tokens.size() && tokens[pos].type=="SEMI") ++pos;
                continue;
            }
            ++pos;
        }
    }
};

void printIR(const std::vector<TACInstruction>& ir,
             const std::string& title) {
    std::cout << "\n=== " << title << " ===\n";
    for (auto& ins : ir) std::cout << "  " << ins.raw << "\n";
    std::cout << "\n";
}

// ==============================================================
// TASK 3: OPTIMIZATION PASSES
// ==============================================================

class Optimizer {
public:
    static bool isNumber(const std::string& s) {
        if (s.empty()) return false;
        size_t st = (s[0]=='-') ? 1 : 0;
        for (size_t i=st; i<s.size(); ++i)
            if (!std::isdigit(s[i])) return false;
        return s.size() > st;
    }

    // 3a. Constant Folding
    static std::vector<TACInstruction>
    constantFolding(std::vector<TACInstruction> ir) {
        for (auto& ins : ir) {
            if (ins.isLabel||ins.isJump||ins.isReturn||
                ins.isLoopHead||ins.isLoopEnd) continue;
            if (!ins.op.empty() && isNumber(ins.arg1) && isNumber(ins.arg2)) {
                long long a=std::stoll(ins.arg1), b=std::stoll(ins.arg2), r=0;
                bool ok=true;
                if (ins.op=="+") r=a+b;
                else if (ins.op=="-") r=a-b;
                else if (ins.op=="*") r=a*b;
                else if (ins.op=="/" && b!=0) r=a/b;
```

```
                else ok=false;
                if (ok) {
                    ins.arg1=std::to_string(r); ins.op=""; ins.arg2=""
    ;
                    ins.raw=ins.result+" = "+ins.arg1;
                }
            }
        }
        return ir;
    }

    // 3b. Constant Propagation
    static std::vector<TACInstruction>
    constantPropagation(std::vector<TACInstruction> ir) {
        std::map<std::string,std::string> cmap;
        for (auto& ins : ir) {
            if (ins.isLabel||ins.isJump||ins.isReturn||
                ins.isLoopHead||ins.isLoopEnd) continue;
            if (cmap.count(ins.arg1)) ins.arg1=cmap[ins.arg1];
            if (cmap.count(ins.arg2)) ins.arg2=cmap[ins.arg2];
            if (ins.op.empty() && isNumber(ins.arg1)) cmap[ins.result
    ]=ins.arg1;
            else cmap.erase(ins.result);
            ins.raw=ins.result+" = "+ins.arg1;
            if (!ins.op.empty()) ins.raw+=" "+ins.op+" "+ins.arg2;
        }
        return ir;
    }

    // 3c. Algebraic Simplification
    static std::vector<TACInstruction>
    algebraicSimplification(std::vector<TACInstruction> ir) {
        for (auto& ins : ir) {
            if (ins.isLabel||ins.isJump||ins.isReturn||
                ins.isLoopHead||ins.isLoopEnd||ins.op.empty())
    continue;
            bool ok=false; std::string na="";
            if (ins.op=="*") {
                if (ins.arg2=="1") {na=ins.arg1;ok=true;}
                else if (ins.arg1=="1") {na=ins.arg2;ok=true;}
                else if (ins.arg2=="0"||ins.arg1=="0") {na="0";ok=true
    ;}
            } else if (ins.op=="+") {
                if (ins.arg2=="0") {na=ins.arg1;ok=true;}
                else if (ins.arg1=="0") {na=ins.arg2;ok=true;}
            } else if (ins.op=="-") {
                if (ins.arg2=="0") {na=ins.arg1;ok=true;}
            } else if (ins.op=="/") {
                if (ins.arg2=="1") {na=ins.arg1;ok=true;}
            }
            if (ok) {
                ins.arg1=na; ins.op=""; ins.arg2="";
                ins.raw=ins.result+" = "+ins.arg1;
            }
        }
        return ir;
    }

```

```
    // 3d. Common Subexpression Elimination
    static std::vector<TACInstruction>
    cse(std::vector<TACInstruction> ir) {
        std::map<std::string,std::string> em;
        for (auto& ins : ir) {
            if (ins.isLabel||ins.isJump||ins.isReturn||
                ins.isLoopHead||ins.isLoopEnd||ins.op.empty())
    continue;
            std::string key=ins.arg1+" "+ins.op+" "+ins.arg2;
            if (em.count(key)) {
                ins.arg1=em[key]; ins.op=""; ins.arg2="";
                ins.raw=ins.result+" = "+ins.arg1;
            } else { em[key]=ins.result; }
        }
        return ir;
    }

    static std::vector<TACInstruction>
    applyAll(std::vector<TACInstruction> ir, bool v=true) {
        if (v) std::cout<<"[Optimizer] Constant Folding...\n";
        ir=constantFolding(ir);
        if (v) std::cout<<"[Optimizer] Constant Propagation...\n";
        ir=constantPropagation(ir);
        if (v) std::cout<<"[Optimizer] Algebraic Simplification...\n";
        ir=algebraicSimplification(ir);
        if (v) std::cout<<"[Optimizer] CSE...\n";
        return cse(ir);
    }
};

// ==============================================================
// TASK 4: DEAD / UNREACHABLE CODE ELIMINATION
// ==============================================================

class DeadCodeEliminator {
public:
    static std::vector<TACInstruction>
    eliminate(std::vector<TACInstruction> ir) {
        std::vector<TACInstruction> result;
        bool dead = false;
        for (auto& ins : ir) {
            if (ins.isLabel) { dead=false; result.push_back(ins);
    continue; }
            if (dead) {
                std::cout<<"  [Dead Code Eliminated]: "<<ins.raw<<"\n"
    ;
                continue;
            }
            result.push_back(ins);
            if (ins.isReturn) dead=true;
        }
        return result;
    }

    static std::vector<TACInstruction>
    deadVarElimination(std::vector<TACInstruction> ir) {
        std::map<std::string,int> uc;
        for (auto& ins : ir) {
```

```
            if (!ins.arg1.empty() && !Optimizer::isNumber(ins.arg1)) uc[ins.arg1]++;
            if (!ins.arg2.empty() && !Optimizer::isNumber(ins.arg2)) uc[ins.arg2]++;
            if (ins.isReturn && !ins.arg1.empty()) uc[ins.arg1]++;
        }
        std::vector<TACInstruction> result;
        for (auto& ins : ir) {
            if (!ins.isLabel&&!ins.isJump&&!ins.isReturn&&
                !ins.isLoopHead&&!ins.isLoopEnd) {
                if (!ins.result.empty() && ins.result[0]=='t' &&
                    uc[ins.result]==0) {
                    std::cout<<"  [Dead Var Eliminated]: "<<ins.raw<<"\n";
                    continue;
                }
            }
            result.push_back(ins);
        }
        return result;
    }
};

// ==============================================================
// TASK 5: LOOP OPTIMIZATIONS
// ==============================================================

class LoopOptimizer {
public:
    // 5a. Loop Invariant Code Motion
    static std::vector<TACInstruction>
    licm(std::vector<TACInstruction> ir) {
        std::vector<TACInstruction> result;
        size_t i = 0;
        while (i < ir.size()) {
            if (!ir[i].isLoopHead) { result.push_back(ir[i++]); continue; }
            std::vector<TACInstruction> body;
            size_t j = i+1;
            while (j<ir.size() && !ir[j].isLoopEnd) body.push_back(ir[j++]);
            std::set<std::string> mod;
            for (auto& ins : body) if (!ins.result.empty()) mod.insert(ins.result);
            std::vector<TACInstruction> hoist, rem;
            for (auto& ins : body) {
                bool inv = !ins.isLabel && !ins.isJump && !ins.op.empty()
                        && !mod.count(ins.arg1)
                        && (ins.arg2.empty()||!mod.count(ins.arg2));
                if (inv) {
                    std::cout<<"  [LICM Hoisted]: "<<ins.raw<<"\n";
                    hoist.push_back(ins); mod.erase(ins.result);
                } else rem.push_back(ins);
            }
            for (auto& h : hoist) result.push_back(h);
            result.push_back(ir[i]);
            for (auto& r : rem)   result.push_back(r);
```

```
            if (j<ir.size()) result.push_back(ir[j]);
            i = j+1;
        }
        return result;
    }

    // 5b. Loop Fusion
    static std::vector<TACInstruction>
    loopFusion(std::vector<TACInstruction> ir) {
        std::vector<TACInstruction> result;
        size_t i = 0;
        while (i < ir.size()) {
            if (!ir[i].isLoopHead) { result.push_back(ir[i++]); continue; }
            std::vector<TACInstruction> b1;
            size_t j = i+1;
            while (j<ir.size() && !ir[j].isLoopEnd) b1.push_back(ir[j++]);
            size_t e1 = j;
            if (e1+1 < ir.size() && ir[e1+1].isLoopHead) {
                std::vector<TACInstruction> b2;
                size_t k = e1+2;
                while (k<ir.size() && !ir[k].isLoopEnd) b2.push_back(ir[k++]);
                std::cout<<"  [Loop Fusion]: Merging "
                         <<ir[i].result<<" and "<<ir[e1+1].result<<"\n";
                TACInstruction hd=ir[i]; hd.raw="LOOP_BEGIN "+hd.result+"_FUSED";
                result.push_back(hd);
                for (auto& b : b1) result.push_back(b);
                for (auto& b : b2) result.push_back(b);
                TACInstruction tl=ir[e1]; tl.raw="LOOP_END "+hd.result+"_FUSED";
                result.push_back(tl);
                i = k+1; continue;
            }
            result.push_back(ir[i]);
            for (auto& b : b1) result.push_back(b);
            if (e1<ir.size()) result.push_back(ir[e1]);
            i = e1+1;
        }
        return result;
    }

    // 5c. Strength Reduction
    static std::vector<TACInstruction>
    strengthReduction(std::vector<TACInstruction> ir) {
        for (auto& ins : ir) {
            if (ins.op=="*" && Optimizer::isNumber(ins.arg2) &&
                std::stoll(ins.arg2)==2) {
                ins.op="+"; ins.arg2=ins.arg1;
                ins.raw=ins.result+" = "+ins.arg1+" + "+ins.arg2;
                std::cout<<"  [Strength Reduction]: mul->add: "<<ins.raw<<"\n";
            }
        }
        return ir;
```

```
    }
};

// ==============================================================
// TASK 6: EXECUTION TIME COMPARISON
// ==============================================================

void benchmarkComparison() {
    std::cout<<"\n=== TASK 6: EXECUTION TIME COMPARISON ===\n";
    const int RUNS=5, N=50000000;
    double sumU=0, sumO=0; long long totU=0, totO=0;
    for (int r=0; r<RUNS; ++r) {
        auto t0=std::chrono::high_resolution_clock::now();
        volatile long long s=0;
        for (int i=0; i<N; ++i) { int f=2; s+=i*f; }
        totU=s;
        auto t1=std::chrono::high_resolution_clock::now();
        sumU+=std::chrono::duration<double,std::milli>(t1-t0).count();
    }
    for (int r=0; r<RUNS; ++r) {
        auto t0=std::chrono::high_resolution_clock::now();
        volatile long long s=0; long long acc=0;
        for (int i=0; i<N; ++i) { s+=acc; acc+=2; }
        totO=s;
        auto t1=std::chrono::high_resolution_clock::now();
        sumO+=std::chrono::duration<double,std::milli>(t1-t0).count();
    }
    double aU=sumU/RUNS, aO=sumO/RUNS;
    std::cout<<std::fixed<<std::setprecision(3);
    std::cout<<"  Unoptimized avg: "<<aU<<" ms\n";
    std::cout<<"  Optimized   avg: "<<aO<<" ms\n";
    std::cout<<"  Speedup        : "<<aU/aO<<"x\n";
    std::cout<<"  (sum="<<totO<<")\n";
}

// ==============================================================
// MAIN
// ==============================================================

int main() {
    std::cout<<"================================================\n";
    std::cout<<" CS-346 Lab 11: Code & Loop Optimization\n";
    std::cout<<" GitHub: github.com/Abrarbut/compiler-construction\n";
    std::cout<<"================================================\n";

    std::string src = R"(
int compute(int a) {
    int x = 3 + 4;
    int y = x * 1;
    int z = y + 0;
    int w = z * 2;
    int t = z * 2;
    return t;
    int dead = 99;
}
)";
    std::string lsrc = R"(
int loopFunc() {
```

```
    int base = 10 * 2;
    for (i = 0; i < 100; i++) {
        sum = sum + i;
        stride = base * 3;
    }
    for (j = 0; j < 100; j++) {
        total = total + j;
    }
    return sum;
}
)";

    Lexer lx; auto toks=lx.tokenize(src); lx.printTokens(toks);

    IRGenerator gen; gen.generateIR(src);
    printIR(gen.instructions,"TASK 2: UNOPTIMIZED IR");

    auto opt=Optimizer::applyAll(gen.instructions);
    printIR(opt,"TASK 3: OPTIMIZED IR (CF+CP+AS+CSE)");

    std::cout<<"\n=== TASK 4: DEAD CODE ELIMINATION ===\n";
    auto clean=DeadCodeEliminator::eliminate(opt);
    clean=DeadCodeEliminator::deadVarElimination(clean);
    printIR(clean,"TASK 4: FINAL CLEAN IR");

    IRGenerator lg; lg.generateIR(lsrc);
    printIR(lg.instructions,"TASK 5: LOOP IR BEFORE");

    std::cout<<"\n=== TASK 5a: LICM ===\n";
    auto lo=LoopOptimizer::licm(lg.instructions);
    std::cout<<"\n=== TASK 5b: LOOP FUSION ===\n";
    lo=LoopOptimizer::loopFusion(lo);
    std::cout<<"\n=== TASK 5c: STRENGTH REDUCTION ===\n";
    lo=LoopOptimizer::strengthReduction(lo);
    printIR(lo,"TASK 5: LOOP IR AFTER");

    benchmarkComparison();

    std::cout<<"\n==================================================\n";
    std::cout<<" Lab 11 Complete.\n";
    std::cout<<"==================================================\n";
    return 0;
}
```

Listing 13: compiler_optimizer.cpp – complete implementation

Program Output

The following output is produced when running the compiled program:

Complete Console Output

```
==================================================
 CS-346 Lab 11: Code & Loop Optimization
 GitHub: github.com/Abrarbut/compiler-construction
==================================================

=== LEXER OUTPUT ===
  [ KEYWORD] int
  [   IDENT] compute
  [   PAREN] (
  [ KEYWORD] int
  [   IDENT] a
  [   PAREN] )
  [   BRACE] {
  [ KEYWORD] int
  [   IDENT] x
  [  ASSIGN] =
  [  NUMBER] 3
  [      OP] +
  [  NUMBER] 4
  [    SEMI] ;
  [ KEYWORD] int
  [   IDENT] y
  [  ASSIGN] =
  [   IDENT] x
  [      OP] *
  [  NUMBER] 1
  [    SEMI] ;
  [ KEYWORD] int
  [   IDENT] z
  [  ASSIGN] =
  [   IDENT] y
  [      OP] +
  [  NUMBER] 0
  [    SEMI] ;
  [ KEYWORD] int
  [   IDENT] w
  [  ASSIGN] =
  [   IDENT] z
  [      OP] *
  [  NUMBER] 2
  [    SEMI] ;
  [ KEYWORD] int
  [   IDENT] t
  [  ASSIGN] =
  [   IDENT] z
```

```
  [      OP] *
  [  NUMBER] 2
  [    SEMI] ;
  [ KEYWORD] return
  [   IDENT] t
  [    SEMI] ;
  [ KEYWORD] int
  [   IDENT] dead
  [  ASSIGN] =
  [  NUMBER] 99
  [    SEMI] ;
  [   BRACE] }

=== TASK 2: UNOPTIMIZED IR ===
  t1 = 3 + 4
  x = t1
  t2 = x * 1
  y = t2
  t3 = y + 0
  z = t3
  t4 = z * 2
  w = t4
  t5 = z * 2
  t = t5
  return t
  dead = 99

[Optimizer] Constant Folding...
[Optimizer] Constant Propagation...
[Optimizer] Algebraic Simplification...
[Optimizer] CSE...

=== TASK 3: OPTIMIZED IR (CF+CP+AS+CSE) ===
  t1 = 7
  x = 7
  t2 = 7
    t2 = 7
  y = t2
  t3 = y
  z = t3
  t4 = z * 2
  w = t4
  t5 = t4
  t = t5
  return t
  dead = 99

=== TASK 4: DEAD CODE ELIMINATION ===
  [Dead Code Eliminated]: dead = 99
  [Dead Var Eliminated]: t1 = 7
```

```
=== TASK 4: FINAL CLEAN IR ===
  x = 7
  t2 = 7
  y = t2
  t3 = y
  z = t3
  t4 = z * 2
  w = t4
  t5 = t4
  t = t5
  return t

=== TASK 5: LOOP IR BEFORE ===
  t1 = 10 * 2
  base = t1
  LOOP_BEGIN LOOP_1
  i = 0
  t2 = sum + i
  sum = t2
  t3 = base * 3
  stride = t3
  LOOP_END LOOP_1
  LOOP_BEGIN LOOP_1
  j = 0
  t4 = total + j
  total = t4
  LOOP_END LOOP_1
  return sum

=== TASK 5a: LICM ===
  [LICM Hoisted]: t3 = base * 3

=== TASK 5b: LOOP FUSION ===
  [Loop Fusion]: Merging LOOP_1 and LOOP_1

=== TASK 5c: STRENGTH REDUCTION ===
  [Strength Reduction]: mul->add: t1 = 10 + 10

=== TASK 5: LOOP IR AFTER ===
  t1 = 10 + 10
  base = t1
  t3 = base * 3
  LOOP_BEGIN LOOP_1_FUSED
  i = 0
  t2 = sum + i
  sum = t2
  stride = t3
  j = 0
  t4 = total + j
```

```
  total = t4
  LOOP_END LOOP_1_FUSED
  return sum

=== TASK 6: EXECUTION TIME COMPARISON ===
  Unoptimized avg: 128.701 ms
  Optimized   avg: 38.991 ms
  Speedup        : 3.287x
  (sum=2499999950000000)

==================================================
 Lab 11 Complete.
==================================================
```

Summary and Conclusions

Task	Technique	Result
1	Modular compiler (5 classes)	Clean pipeline
2	TAC IR generation	Correct three-address code
3a	Constant Folding	`3+4` → `7` at compile time
3b	Constant Propagation	Variable constants substituted
3c	Algebraic Simplification	`x*1`, `x+0` removed
3d	CSE	Duplicate `z*2` reused
4a	Unreachable Elimination	Code after `return` dropped
4b	Dead Variable Elimination	Unused temporaries dropped
5a	LICM	Invariant hoisted outside loop
5b	Loop Fusion	Two loops merged into one
5c	Strength Reduction	Multiply replaced with add
6	Benchmark	$\approx 3.3\times$ speedup measured

Key Observations

1. **Constant Folding** eliminated unnecessary runtime computations by evaluating `3+4` at compile time.
2. **Constant Propagation** successfully propagated the value `7` throughout the IR, enabling further simplifications.
3. **Algebraic Simplification** removed identity operations like `x*1` and `y+0`, reducing instruction count.

4. **Common Subexpression Elimination** identified that `z*2` was computed twice and reused the first result (`t4`).
5. **Unreachable Code Elimination** correctly removed the statement `dead = 99` because it appeared after a `return`.
6. **Dead Variable Elimination** removed temporary `t1` which was never used after its definition.
7. **Loop Invariant Code Motion** hoisted `base * 3` outside the loop, eliminating 100 redundant multiplications.
8. **Loop Fusion** merged two identical loops, reducing loop overhead by 50% and improving cache locality.
9. **Strength Reduction** replaced `10 * 2` with `10 + 10`, demonstrating that addition is cheaper than multiplication.
10. The **benchmark** showed a **3.287x speedup** when strength reduction was applied to replace multiplications with additions in a tight loop.

Conclusion

Even a sequence of simple, local passes collectively yields significant improvements. The same techniques — backed by full data-flow analysis — power production compilers like GCC and Clang at `-O1`, `-O2`, and `-O3` optimization levels.

The modular architecture implemented in this lab demonstrates how modern compilers are organized into clear, reusable passes. Each optimization preserves program semantics while improving performance metrics. The **3.287x speedup** in the benchmark validates that even simple optimizations can have dramatic effects on loop-intensive code.